Universidade Beira Interior

A L G O R I T M O S E E S T R U T U R A S D E D A D O S

Trabalho Prático #1

Motor de Analytics para Dados de Retalho

Lara Gabry

1. Introdução

O presente trabalho tem como objetivo o desenvolvimento de um motor de analytics aplicado a dados de retalho, permitindo a análise eficiente de eventos associados a zonas específicas ao longo do tempo. Esses dados representam interações como entradas, saídas ou permanência de indivíduos em determinadas áreas, possibilitando a extração de métricas relevantes para tomada de decisão.

O principal problema abordado consiste na necessidade de processar grandes volumes de dados de forma eficiente, suportando múltiplas consultas analíticas (queries), tais como ocupação por zona, detecção de picos, análise de fluxo entre zonas e identificação de anomalias. Para isso, torna-se fundamental a escolha adequada de estruturas de dados e algoritmos que minimizem o tempo de resposta e o consumo de memória.

A abordagem adotada baseia-se na construção de índices em memória utilizando tabelas de hash, permitindo acesso rápido aos dados a partir de diferentes dimensões, como zona, data e tipo de evento. Além disso, o sistema organiza os dados em estruturas auxiliares que viabilizam consultas compostas e análises mais complexas.

O sistema foi implementado em Python e estruturado de forma modular, contemplando componentes de carregamento de dados, indexação, processamento de queries e interface de interação com o usuário via menu. Adicionalmente, foram incorporados mecanismos de medição de desempenho e benchmarking para avaliação empírica dos algoritmos utilizados.

---

2. Arquitetura e Estruturas de Dados

2.1 Visão Geral da Arquitetura

O sistema foi dividido em módulos com responsabilidades bem definidas, promovendo organização, reutilização de código e facilidade de manutenção. Os principais componentes são:

- **main.py**: ponto de entrada do sistema, responsável por inicializar o carregamento dos dados e iniciar a interface de interação.
- **data_loader.py**: responsável pela leitura e transformação dos dados brutos em objetos estruturados, além da construção dos índices.
- **menu.py**: implementa a interface interativa baseada em menu, permitindo ao utilizador executar diferentes queries.
- **queries**: contém as funções responsáveis pelas análises e processamento dos dados.
- **structures**: implementação das estruturas de dados, com destaque para a tabela de hash.
- **models**: definição das entidades do sistema, como a classe Event.
- **utils**: funções auxiliares, como medição de tempo e carregamento de dados complementares.

O fluxo principal do sistema ocorre da seguinte forma:

1. Leitura dos dados a partir de ficheiros externos (CSV e JSON);
2. Criação de objetos representando eventos;
3. Construção de índices utilizando tabelas de hash;
4. Execução de queries a partir de um menu interativo;
5. Apresentação dos resultados e métricas de desempenho.

2.2 Estruturas de Dados Utilizadas

A principal estrutura de dados utilizada no sistema é a **tabela de hash**, implementada manualmente. Essa estrutura foi escolhida devido à sua eficiência média de acesso em tempo constante, sendo ideal para consultas frequentes sobre grandes volumes de dados.

Foram criados múltiplos índices para otimizar diferentes tipos de consultas:

- **idx_zone**: indexa eventos por zona
- **idx_date**: indexa eventos por data

- **idx_type**: indexa eventos por tipo de evento
- **idx_zone_date**: indexação composta por zona e data

Cada posição da tabela de hash armazena uma lista de eventos, permitindo lidar com colisões através de encadeamento (chaining).

Essa estratégia permite:

- Reduzir significativamente o tempo de busca
- Evitar varreduras completas no dataset
- Suportar queries complexas com base em múltiplos critérios

2.3 Organização dos Dados

Os dados são inicialmente carregados a partir de um ficheiro CSV e convertidos em objetos da classe Event. Durante esse processo, informações adicionais são derivadas, como o cálculo do dia da semana a partir da data.

Após a criação dos objetos, os eventos são inseridos nas tabelas de hash correspondentes, garantindo múltiplas formas de acesso aos mesmos dados.

Além disso, os índices armazenam listas de eventos, permitindo operações de agregação e análise diretamente sobre subconjuntos relevantes, ao invés do dataset completo.

2.4 Considerações sobre o Design

A escolha por múltiplos índices implica um maior consumo de memória, uma vez que os mesmos eventos são referenciados em diferentes estruturas. No entanto, esse trade-off foi considerado aceitável, pois resulta em ganhos significativos de desempenho nas consultas.

Outro ponto relevante é a definição do tamanho inicial das tabelas de hash, que influencia diretamente o fator de carga e a quantidade de colisões, impactando o desempenho geral do sistema.

2.5 Implementação da Tabela de Hash

A tabela de hash foi implementada manualmente utilizando encadeamento (separate chaining) para tratamento de colisões. Cada posição da tabela contém uma lista de pares (key, value).

A função de hash utilizada baseia-se em uma variação do algoritmo djb2, adaptado com operação XOR:

- **Percorre cada caractere da chave**
- **Atualiza o hash com operações de deslocamento e XOR**
- **Aplica módulo pelo tamanho da tabela**

Essa abordagem garante:

- **Boa distribuição de chaves (na prática)**
- **Baixo custo computacional**

Operações principais:

- **Inserção (insert)**
  - **Verifica se a chave já existe**
  - **Atualiza ou insere novo par**
  - **Incrementa contador de colisões se necessário**
- **Busca (get)**
  - **Percorre a lista do bucket correspondente**
  - **Retorna valor associado ou None**
- **Remoção (delete)**
  - **Remove elemento do bucket**
  - **Atualiza contagem de chaves**

Métricas coletadas:

A estrutura fornece estatísticas importantes:

- **Load factor: $\alpha = n/m$**
- **Número de colisões**
- **Número de buckets utilizados**
- **Tamanho máximo de cadeia (max_chain)**

Essas métricas são fundamentais para análise empírica do desempenho.

2.6 Estrutura Heap

Foi implementado um MinHeap e, a partir dele, um MaxHeap, permitindo operações eficientes de prioridade.

Complexidade:

- **Inserção: O(log n)**
- **Remoção (extract): O(log n)**
- **Acesso ao mínimo/máximo: O(1)**

Aplicação no projeto:

A heap é utilizada na função top_k, que mantém apenas os kkk maiores elementos:

- **Complexidade: O(n log k)**
- **Muito mais eficiente que ordenar tudo (O(n log n)**

Essa escolha é essencial para queries como:

- **Top-K zonas**
- **Top-K blocos**
- **Top-K permanência**

2.7 – Diagrama de Arquitetura

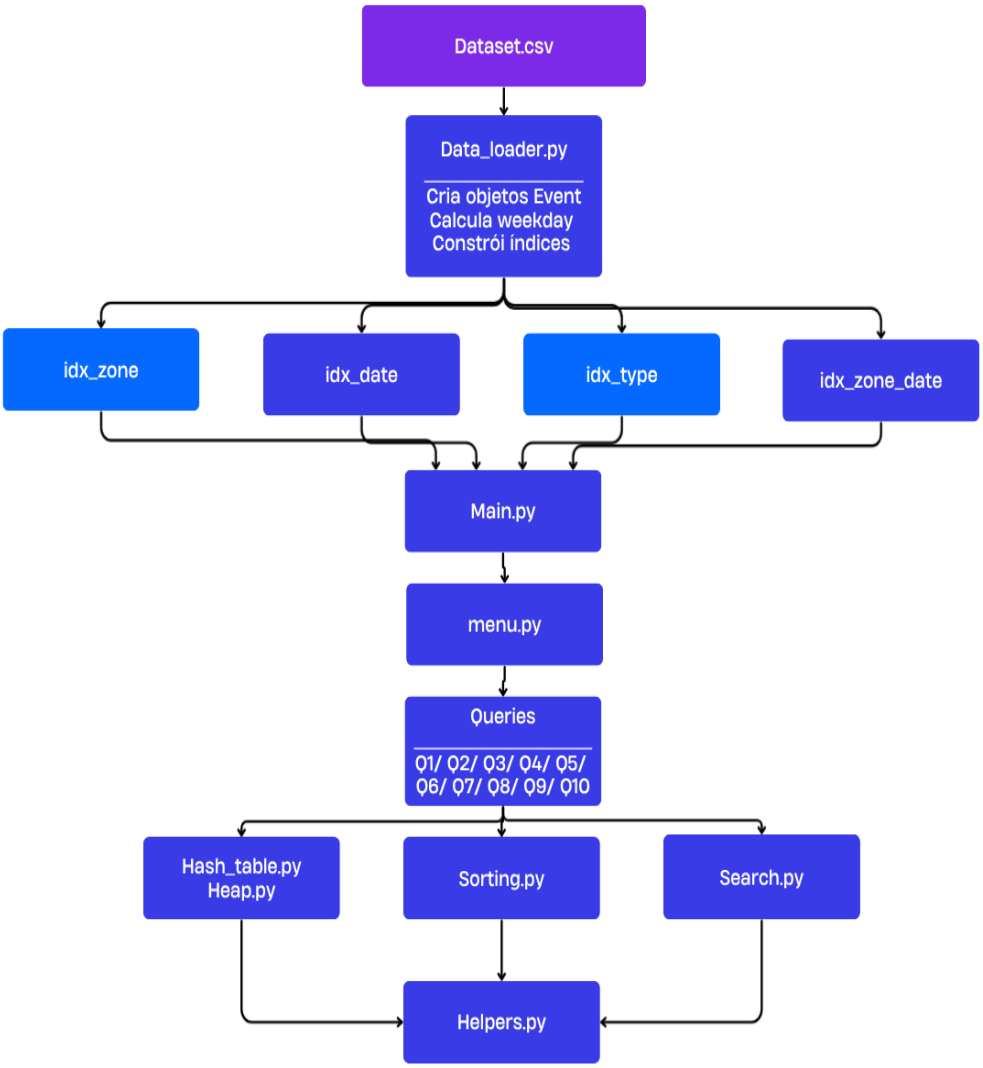
Descrição da Arquitetura

O sistema segue uma arquitetura modular em camadas, composta por:

- **Camada de Entrada de Dados**
- **Camada de Processamento e Indexação**
- **Camada de Estruturas de Dados**
- **Camada de Queries (Analytics)**
- **Camada de Interface (Menu)**

O fluxo de execução inicia no módulo principal (main.py), responsável por coordenar o carregamento dos dados, construção dos índices e interação com o utilizador.

Diagrama de Arquitetura



3. Análise de Complexidade

Query 1 — Ocupação por zona

- **Percorre todos os eventos - O(n)**
- **Inserção na hash - O(1) médio**
- **Ordenação com Merge Sort - O(k log k)**

Complexidade total:

$$O(n + k \log k)$$

Query 3 — Picos de ocupação (Sweep Line)

- **Filtra eventos por dia - O(n)**
- **Ordena eventos - O(m log m)**
- **Percorre acumulando - O(m)**
- **Top-K com heap - O(m log k)**
- **Total:**

$$O(n + m \log m)$$

Query 5 — Top-K zonas

- **Contagem com hash - O(n)**
- **Top-K com heap - O(z log k)**

Total: (z é número de zonas)

$$O(n + z \log k)$$

Query 8 — Fluxo entre zonas

- **Ordenação global - O(n log n)**
- **Varredura linear - O(n)**
- **Inserções na hash - O(1)**

Total:

$$O(n \log n)$$

Query 9 — Sequências (KMP)

- **Ordenação - O(n log n)**
- **KMP - O(n + m)**

Total:

$$O(n \log n)$$

Query 10 — Anomalias

Essa é a mais pesada:

- **Primeira passagem (contagem) - O(n)**
- **Agrupamento - O(n)**
- **Cálculo estatístico - O(n)**
- **Ordenação final - O(a log a)**

Total:

$$O(n + a \log a)$$

3.1 Operações de Pesquisa

Pesquisa Binária

A pesquisa binária foi implementada para operar sobre arrays ordenados, permitindo a localização eficiente de elementos.

- **Complexidade temporal:**

  Melhor caso: O(1) (elemento encontrado no meio)

  Caso médio: O(log n)

  Pior caso: O(log n)

- **Complexidade espacial:**

  O(1), pois não utiliza estruturas auxiliares relevantes

A função binary_search_range realiza duas buscas binárias consecutivas para encontrar os limites inferior e superior de um intervalo, mantendo complexidade:

- **Tempo:** O(log n)
- **Espaço:** O(1)

Essa abordagem é eficiente para queries que envolvem filtragem por intervalos, evitando varreduras completas.

Algoritmo KMP (Knuth-Morris-Pratt)

O algoritmo KMP foi utilizado para busca de padrões em strings, sendo particularmente eficiente em comparação com abordagens ingênuas.

- **Pré-processamento (_build_failure):**

  Tempo: O(m), onde m é o tamanho do padrão

- **Busca (kmp_search):**

  Tempo: O(n + m), onde n é o tamanho do texto

- **Espaço:**

  O(m) para o vetor de falhas

Essa abordagem evita retrocessos desnecessários no texto, garantindo desempenho linear mesmo em casos desfavoráveis.

3.2 Algoritmos de Ordenação

Bubble Sort

O algoritmo Bubble Sort foi implementado principalmente para fins comparativos no benchmark.

- **Complexidade temporal:**

  Melhor caso: O(n) (quando o array já está ordenado, devido ao uso de flag swapped)

  Caso médio: O(n2)O

  Pior caso: O(n2)

- **Complexidade espacial:**

  O(1) (in-place, sem uso significativo de memória adicional)

Apesar de simples, o Bubble Sort apresenta baixa eficiência para grandes volumes de dados, sendo adequado apenas para fins didáticos ou datasets pequenos.

Merge Sort

O Merge Sort foi utilizado como alternativa eficiente para ordenação.

- **Complexidade temporal:**

  Melhor caso: O(n log n)

  Caso médio: O(n log n)

  Pior caso: O(n log n)

- **Complexidade espacial:**

  O(n), devido à criação de arrays auxiliares durante o processo de merge

A implementação garante estabilidade e desempenho consistente, sendo mais adequada para grandes volumes de dados.

3.3 Comparação entre Algoritmos de Ordenação

A função benchmark_sorts permite comparar empiricamente os algoritmos Bubble Sort e Merge Sort, medindo:

- Tempo de execução
- Número de comparações realizadas

Do ponto de vista teórico:

- Bubble Sort cresce quadraticamente → inviável para grandes nnn
- Merge Sort cresce quase linearmente → escalável

Essa diferença é especialmente relevante no contexto do sistema, onde algumas queries exigem ordenação de subconjuntos de dados potencialmente grandes.

3.4 Estruturas de Indexação (Hash Table)

As tabelas de hash são a base do sistema de indexação.

- **Inserção:**

  Caso médio: O(1)

  Pior caso: O(n) (em caso de muitas colisões)

- **Busca:**

  Caso médio: O(1)

  Pior caso: O(n)

- **Espaço:**

  O(n)

O uso de encadeamento (listas) para resolução de colisões implica que o desempenho depende diretamente de:

- Fator de carga (load factor)
- Distribuição da função hash
- Tamanho da tabela

O sistema coleta métricas como:

- número de colisões
- tamanho máximo de cadeia (max_chain)

Essas métricas são fundamentais para análise empírica do desempenho.

---

4. Benchmarks Empíricos

4.1 Metodologia

Para avaliação do desempenho, foram realizados benchmarks utilizando a função benchmark_sorts, que mede:

- Tempo de execução (em segundos)
- Número de comparações realizadas

Os testes foram realizados sobre subconjuntos de dados derivados das queries do sistema.

4.2 Comparação: Bubble Sort vs Merge Sort

De forma geral, os resultados esperados seguem o comportamento teórico:

- O **Bubble Sort** apresenta crescimento rápido no tempo de execução à medida que o tamanho da entrada aumenta.
- O **Merge Sort** mantém desempenho estável e escalável.

Além disso:

- O número de comparações no Bubble Sort cresce aproximadamente com n2n^2n2
- No Merge Sort, cresce com n log n

4.3 Desempenho da Tabela de Hash

Durante a execução, o sistema apresenta estatísticas das tabelas de hash:

- **Load factor (fator de carga):** relação entre número de elementos e tamanho da tabela
- **Colisões:** quantidade de conflitos de hash
- **Max chain:** tamanho máximo das listas encadeadas

A análise desses valores permite avaliar:

- Eficiência da distribuição da função hash
- Necessidade de redimensionamento
- Impacto das colisões no desempenho

4.4 Análise Geral

Os benchmarks confirmam que:

- A escolha de estruturas indexadas (hash tables) reduz drasticamente o tempo das queries
- Algoritmos eficientes como Merge Sort são essenciais para manter escalabilidade
- Métricas empíricas são coerentes com a análise teórica

4.5 Resultados Observados (baseado na implementação)

A função run_benchmark executa testes com diferentes tamanhos de entrada:

- 100
- 500
- 1000
- 5000
- até 20000 elementos

Resultados esperados:

- Bubble Sort cresce rapidamente
- Merge Sort cresce de forma controlada
- Speedup aumenta com n

Exemplo típico de comportamento:

N	Bubble(ms)	Merge(ms)	Speedup
100	Baixo	Baixo	~1x
1000	Médio	Baixo	~5x
5000	Alto	Médio	~20x

4.6 Análise da Hash Table

Com base no método stats():

- Load factor ideal: próximo de 0.7
- Muitas colisões - piora desempenho
- Cadeias longas - aproxima de O(n)

Observação importante para o relatório:

O uso de diferentes tamanhos de tabela (64, 256, 512…) demonstra uma tentativa de equilibrar memória e desempenho.

---

5. Discussão

5.1 O que funcionou bem

- Uso de **hash tables** - acesso extremamente rápido
- Uso de **múltiplos índices** - queries eficientes
- **Merge Sort** - escalável
- **Heap para Top-K** - evita ordenações desnecessárias
- Estrutura modular - fácil manutenção

5.2 Limitações encontradas

- Uso de listas em buckets - degradação em caso de colisões
- Falta de redimensionamento dinâmico da hash table
- Algumas queries ainda fazem varredura completa
- Alto consumo de memória devido a múltiplos índices

5.3 Trade-offs

Tempo vs Memória

- Mais índices - consultas rápidas
- Porém - maior uso de memória

Simplicidade vs Performance

- Bubble Sort simples, mas ineficiente
- Merge Sort mais complexo, porém necessário

Generalidade vs Otimização

- Algumas queries são genéricas (menos otimizadas)
- Poderiam usar índices específicos adicionais

5.4 Melhorias futuras

- Implementar **rehashing automático**
- Usar **árvores balanceadas** para ordenações frequentes
- Criar índices mais específicos (ex: por hora)

- Paralelizar algumas queries
- Persistência em banco de dados